

PRACTICAL-4 DAA LAB

Name- Sarthak Nimbalkar

Class- A8-B4

Roll No- 56

Aim –

Implement maximum sum of subarray for the given scenario of resource allocation using the divide and conquer approach. Problem Statement: A project requires allocating resources to various tasks over a period of time. Each task requires a certain amount of resources, and you want to maximize the overall efficiency of resource usage. You're given an array resources where resources[i] represents the amount of resources required for the ith task. Your goal is to find the contiguous subarray of tasks that maximizes the total resources utilized without exceeding a given resource constraint. Handle cases where the total resources exceed the constraint by adjusting the subarray window accordingly. Your implementation should handle various cases, including scenarios where there's no feasible subarray given the constraint and scenarios where multiple subarrays yield the same maximum resource utilization.

Code

```
#include <stdio.h>
```

```
typedef struct {
```

```
    int total;
```

```
    int from;
```

```
    int to;
```

```
} Segment;
```

```
Segment Find_Max_Segment(int arr[], int left, int right, int limit) {
```

```
    Segment invalid = {0, -1, -1};
```

```
    if (left > right) return invalid;
```

```
    if (left == right) {
```

```
        if (arr[left] <= limit) {
```

```
            return (Segment){arr[left], left, right};
```

```
        } else {
```

```
            return invalid;
```

```
        }
```

```
    }
```

```
    int middle = (left + right) / 2;
```

```
    Segment segLeft = Find_Max_Segment(arr, left, middle, limit);
```

```
    Segment segRight = Find_Max_Segment(arr, middle + 1, right, limit);
```

```
    int tempSum = 0, maxLeft = 0, startIdx = middle;
```

```
    for (int i = middle; i >= left; i--) {
```

```
        tempSum += arr[i];
```

```
        if (tempSum <= limit && tempSum > maxLeft) {
```

```
            maxLeft = tempSum;
```

```

        startIdx = i;
    }
}

tempSum = 0;
int maxRight = 0, endIdx = middle + 1;
for (int j = middle + 1; j <= right; j++) {
    tempSum += arr[j];
    if (tempSum <= limit && tempSum > maxRight) {
        maxRight = tempSum;
        endIdx = j;
    }
}

int combinedSum = maxLeft + maxRight;
Segment segCross = {0, -1, -1};
if (combinedSum <= limit) {
    segCross.total = combinedSum;
    segCross.from = startIdx;
    segCross.to = endIdx;
}

if (segLeft.total >= segRight.total && segLeft.total >= segCross.total)
    return segLeft;
else if (segRight.total >= segLeft.total && segRight.total >= segCross.total)

```

```
        return segRight;
    else
        return segCross;
}
```

```
int main() {
    int size;

    printf("Size of Resource Array: ");
    scanf("%d", &size);

    if (size <= 0) {
        printf("Empty array. Exiting.\n");
        return 0;
    }

    int resources[size];
    printf("Input the Resource Array:\n");
    for (int i = 0; i < size; i++) {
        printf("Element %d: ", i + 1);
        scanf("%d", &resources[i]);
    }

    int maxAllowed;
    printf("Enter constraint limit: ");
```

```
scanf("%d", &maxAllowed);
```

```
Segment answer = Find_Max_Segment(resources, 0, size - 1, maxAllowed);
```

```
if (answer.total == 0) {
```

```
    printf("No suitable subarray found.\n");
```

```
} else {
```

```
    printf("Maximum Sum = %d, between indices %d and %d\n", answer.total,  
answer.from, answer.to);
```

```
}
```

```
return 0;
```

```
}
```

OUTPUTS

ACC TO QUESTIONS

1. Basic small array

Output Clear

```
Size of Resource Array: 4  
Input the Resource Array:  
Element 1: 2  
Element 2: 1  
Element 3: 3  
Element 4: 4  
Enter constraint limit: 5  
Maximum Sum = 4, between indices 3 and 3  
  
=== Code Execution Successful ===
```

2. Exact match to constraint

Output

Clear

```
Size of Resource Array: 4
Input the Resource Array:
Element 1: 2
Element 2:
2
Element 3: 2
Element 4: 2
Enter constraint limit: 4
Maximum Sum = 4, between indices 0 and 1

=== Code Execution Successful ===
```

3. Single element equals constraint

Output

Clear

```
Size of Resource Array: 4
Input the Resource Array:
Element 1: 1
Element 2: 5
Element 3: 2
Element 4: 3
Enter constraint limit: 5
Maximum Sum = 5, between indices 1 and 1

=== Code Execution Successful ===
```

4. All elements smaller but no combination fits

Output

Clear

```
Size of Resource Array: 3
Input the Resource Array:
Element 1: 6
Element 2: 7
Element 3: 8
Enter constraint limit: 5
No suitable subarray found.

=== Code Execution Successful ===
```

5. Multiple optimal subarrays

main.c

Output



```
Size of Resource Array: 5
Input the Resource Array:
Element 1: 1
Element 2: 2
Element 3: 3
Element 4: 2
Element 5: 1
Enter constraint limit: 5
Maximum Sum = 3, between indices 0 and 1

=== Code Execution Successful ===
```

6. Large window valid

main.c

Output



```
Size of Resource Array: 4
Input the Resource Array:
Element 1: 1
Element 2: 1
Element 3: 1
Element 4: 1
Enter constraint limit: 4
Maximum Sum = 4, between indices 0 and 3

=== Code Execution Successful ===
```

7. Sliding window shrink needed

main.c

Output



```
Size of Resource Array: 4
Input the Resource Array:
Element 1: 4
Element 2: 2
Element 3: 3
Element 4: 1
Enter constraint limit: 5
Maximum Sum = 4, between indices 0 and 0

=== Code Execution Successful ===
```

8. Empty array

main.c

Output



```
Size of Resource Array: 0
Empty array. Exiting.

=== Code Execution Successful ===
```

9. Constraint = 0

main.c

Output



```
Size of Resource Array: 3
Input the Resource Array:
Element 1: 1
Element 2: 2
Element 3: 3
Enter constraint limit: 0
No suitable subarray found.

=== Code Execution Successful ===
```